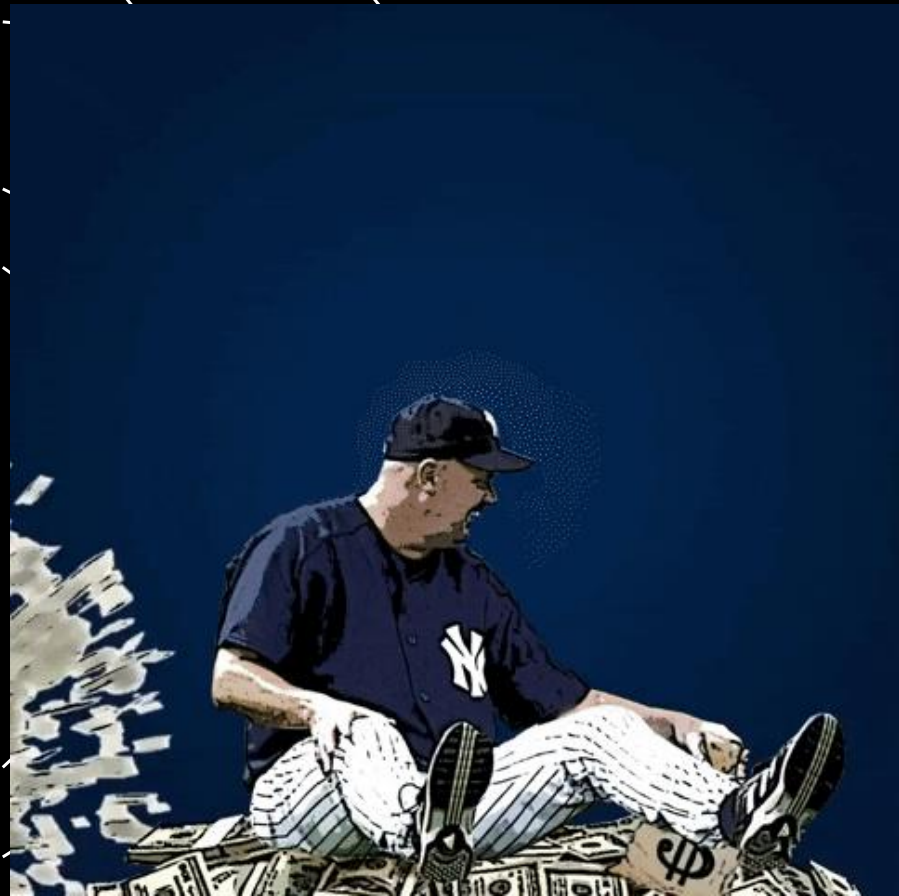


DOES TEAM AFFECT HOME
RUNS MLB PLAYERS HIT?

WHAT BATTING METRICS
AFFECT MLB PLAYERS
SALARY?

Jessica Lamastra
ECO 590 project



RESEARCH QUESTIONS

How does team influence home runs?

I wanted to research this question because people always argue that some teams hit more home runs than others because of the stadium dimensions.

Variables needed:

- Team
- Position
- Home Run
- War

Do teams value traditional or advanced batting metrics when determining Salary?

There are more home run hitters that are signing larger multi million-dollar contracts. I wanted to see how teams determine these contracts.

Variables needed:

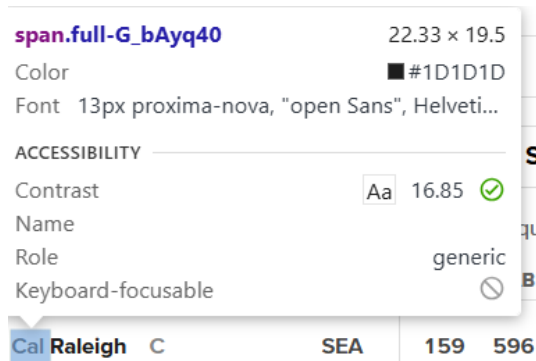
- OPS
- WAR
- RBI
- Batting Average
- Home Run
- Position

I scraped data from 3 different sites, ESPN, MLB, and USA Today using BeautifulSoup and Selenium.

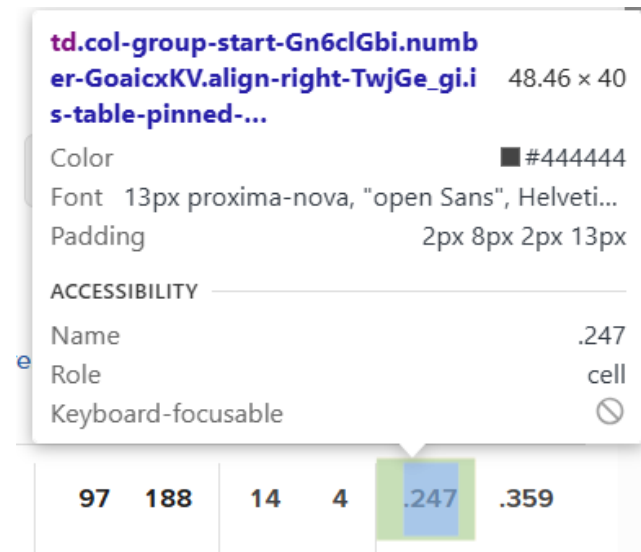
All data is gathered from 2025 completed baseball regular season

GATHERING DATA

- All information found by reputable sources (needed the data accurate)
- Combination of BeautifulSoup and Selenium because some I could scrape via tags and get requests others I had to use Xpath because of java code
- Not all data I needed was on 1 website
- Used tags a and td
- For players I filtered by span to get all elements with class specified and dropped the names into a list



```
<a class="bui-link" href="/player/663728" tabindex="0"
aria-label="Cal Raleigh">
  <span class="full-G_bAyq40">...</span>
  <span class="short-IiSPVSQp">C Raleigh</span>
```



Just based on this code I could not parse through and scrape with BeautifulSoup so I switched to Selenium

Class A and tag bui-link worked

GATHERING DATA WITH BEAUTIFULSOUP

```
#players name in mlb
players = [] #creates empty list called players
homers = [] #creates empty homers list
for i in range(1, 32):
    mlb_url1 = f"https://www.mlb.com/stats/2025?page={i}" #loops through all 31 pages on mlb site

# Use requests to retrieve data from a given URL
mlb_response1 = requests.get(mlb_url1)
# Parse the whole HTML page using BeautifulSoup
mlb_soup1 = BeautifulSoup(mlb_response1.text, 'html.parser')

# find all player and homers tag and put them in a list
player_links = mlb_soup1.find_all('a', class_='bui-link') #find all a tags on the page that have class bui-link
tag_hr = mlb_soup1.find_all('td', {'class':'selected-h6IPIIxg number-GoaicxKV align-right-TwjGe_gi is-table-pinned-lGP8KWTk'}) #get the tags

for link in player_links: #loops through each tag found
    # find first and last name spans inside this <a>
    spans = link.find_all('span', class_='full-G_bAyq40') #finds all elemnets with class specified
    if len(spans) >= 2: # make sure we have first and last name (2 elements in link)
        first_name = spans[0].get_text(strip=True)
        last_name = spans[1].get_text(strip=True)
        #indicated first span is first name and second span is last name
        full_name = f"{first_name} {last_name}" #creates 1 variable with first and last name
        players.append(full_name) #appends names to a list called players

for link in tag_hr: # for each link in the list tags
    x1 = link.get_text() # get the text within the node in that list item link
    homers.append(x1) #append it to the list called homers
```

GATHERING DATA - SELENIUM

- Used Selenium chrome webdriver
- Waited 7 seconds for elements to load
- Found elements using Xpath by finding all rows in table
- Code specifies to only grab certain elements in table and drops it into corresponding list for all 31 pages of site

```
mlb_rbis = []
mlb_average = []
mlb_ops = []

for i in range(1, 32):
    mlb_url1 = f"https://www.mlb.com/stats/2025?page={i}"

    driver.get(mlb_url1) #Uses Selenium's Chrome WebDriver to open the page in a browser
    driver.implicitly_wait(7) #Tells Selenium to wait up to 7 seconds for elements to load before throwing an error
    rows = driver.find_elements(By.XPATH, '//*[@id="stats-app-root"]/section/section/div[4]/div[1]/div/table/tbody/tr')

    for row in rows: #Loops through each row in table (each player)
        try:
            rbi = row.find_element(By.XPATH, './td[9]').text #Looks inside the current row (.) for the 9th <td> element
            mlb_rbis.append(int(rbi)) #Converts the RBI value from a string to an integer and adds the number to mlb_rbi

            avg = row.find_element(By.XPATH, './td[14]').text #Looks inside the current row for the 14th element (the ba
            mlb_average.append(float(avg)) #appends to the list mlb_average as a float instead of string

            ops = row.find_element(By.XPATH, './td[17]').text #Looks inside the current row for the 17 element (ops) and
            mlb_ops.append(float(ops)) #appends the number as a float to list mlb_ops
        except:
            continue

    #Inside a try block to avoid errors if a row doesn't have an RBI value.
    # except/continue: If there's an error (e.g., RBI column missing, text not an integer), it skips that row and continues
```

DATA FRAME

I put all lists collected from 1 website in data frame

- Matched element 1 of all lists together for all 765 elements
- Repeated this web scraping process for sites 2 and 3

```
df1 = pd.DataFrame({ #creates new pandas dataframe
    'Player': players, #creates column called Player
    'OPS': mlb_ops, #creates column called OPS
    'HR': homers, #creates column called HR and
    'BattingAvg': mlb_average, #creates column
    'RBI': mlb_rbis #creates column called RBI
})
```

*#I did this way because all my lists are from the same website
#The index alignment is by list order (first element of each list)
#df1 is a structured table of player stats that*

```
print(df1.head())
print(f'The length of df1 is {len(df1)}')
```

	Player	OPS	HR	BattingAvg	RBI
0	Cal Raleigh	0.948	60	0.247	125
1	Kyle Schwarber	0.928	56	0.240	132
2	Shohei Ohtani	1.014	55	0.282	102
3	Aaron Judge	1.145	53	0.331	114
4	Eugenio Suárez	0.824	49	0.228	118

The length of df1 is 765

MERGING

- Merged my 3 data frames into 2 CSVs (one is a lot larger than another)
- Merged by “Player” so all data corresponding to the same player would be in one row
- Only kept players that were in both data frames so some were dropped
- Used descriptive statistics to check my data frames merged properly

```
merged_df = pd.merge(df1, df2, on=["Player"], how="inner")

print(merged_df.head())
print(len(merged_df))
print(merged_df.describe(include='all')) #this gives me descriptive statistics
```

```
merged_df2 = pd.merge(merged_df, df3, on="Player", how="inner")

print(merged_df2.head())
print(len(merged_df2))
print(merged_df2.describe(include='all'))
```

MERGED DESCRIBED DATA FROM DATA1

	Player	OPS	HR	BattingAvg	RBI	Position \
count	431	431.000000	431	431.000000	431.000000	431
unique	428	NaN	44	NaN	NaN	11
top	Max Muncy	NaN	0	NaN	NaN	OF
freq	4	NaN	94	NaN	NaN	127
mean	NaN	0.593599	NaN	0.204327	36.983759	NaN
std	NaN	0.267535	NaN	0.096299	31.321472	NaN
min	NaN	0.000000	NaN	0.000000	0.000000	NaN
25%	NaN	0.554000	NaN	0.191000	8.000000	NaN
50%	NaN	0.670000	NaN	0.233000	32.000000	NaN
75%	NaN	0.752000	NaN	0.257500	61.000000	NaN
max	NaN	2.000000	NaN	1.000000	132.000000	NaN

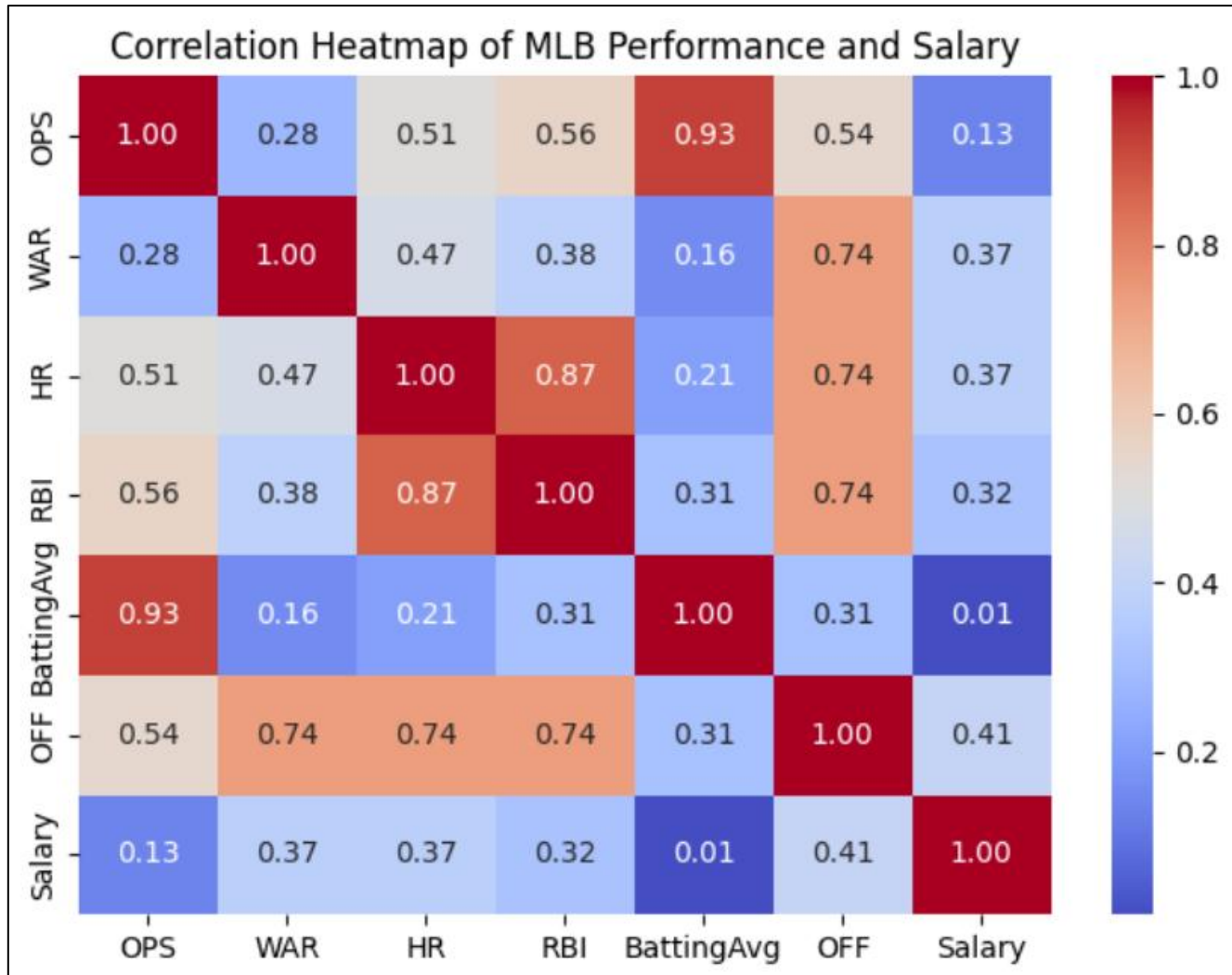
	Team	Salary
count	431	4.310000e+02
unique	30	NaN
top	Diamondbacks	NaN
freq	18	NaN
mean	NaN	5.425316e+06
std	NaN	8.191519e+06
min	NaN	7.600000e+05
25%	NaN	7.723500e+05
50%	NaN	1.350000e+06
75%	NaN	6.000000e+06
max	NaN	6.187500e+07

MERGED DESCRIBED DATA FROM DATA2

	Player	OPS	HR	BattingAvg	RBI	Position	Team
count	115	115.000000	115	115.000000	115.000000	115	115
unique	114	NaN	38	NaN	NaN	11	29
top	Max Muncy	NaN	0	NaN	NaN	OF	Cubs
freq	2	NaN	9	NaN	NaN	35	9
mean	NaN	0.746922	NaN	0.250765	67.530435	NaN	NaN
std	NaN	0.246031	NaN	0.099890	29.517857	NaN	NaN
min	NaN	0.000000	NaN	0.000000	0.000000	NaN	NaN
25%	NaN	0.729500	NaN	0.244500	53.000000	NaN	NaN
50%	NaN	0.776000	NaN	0.261000	70.000000	NaN	NaN
75%	NaN	0.828500	NaN	0.276000	86.500000	NaN	NaN
max	NaN	2.000000	NaN	1.000000	132.000000	NaN	NaN

	Salary	OFF	WAR
count	1.150000e+02	115.000000	115.000000
unique	NaN	NaN	NaN
top	NaN	NaN	NaN
freq	NaN	NaN	NaN
mean	9.772414e+06	3.220000	3.742609
std	1.106592e+07	1.668028	1.368381
min	7.600000e+05	-0.000000	2.100000
25%	7.805500e+05	2.300000	2.700000
50%	5.000000e+06	3.200000	3.500000
75%	1.679545e+07	4.150000	4.500000
max	6.187500e+07	9.400000	9.700000

HOW CONTINUOUS DATA CORRELATES

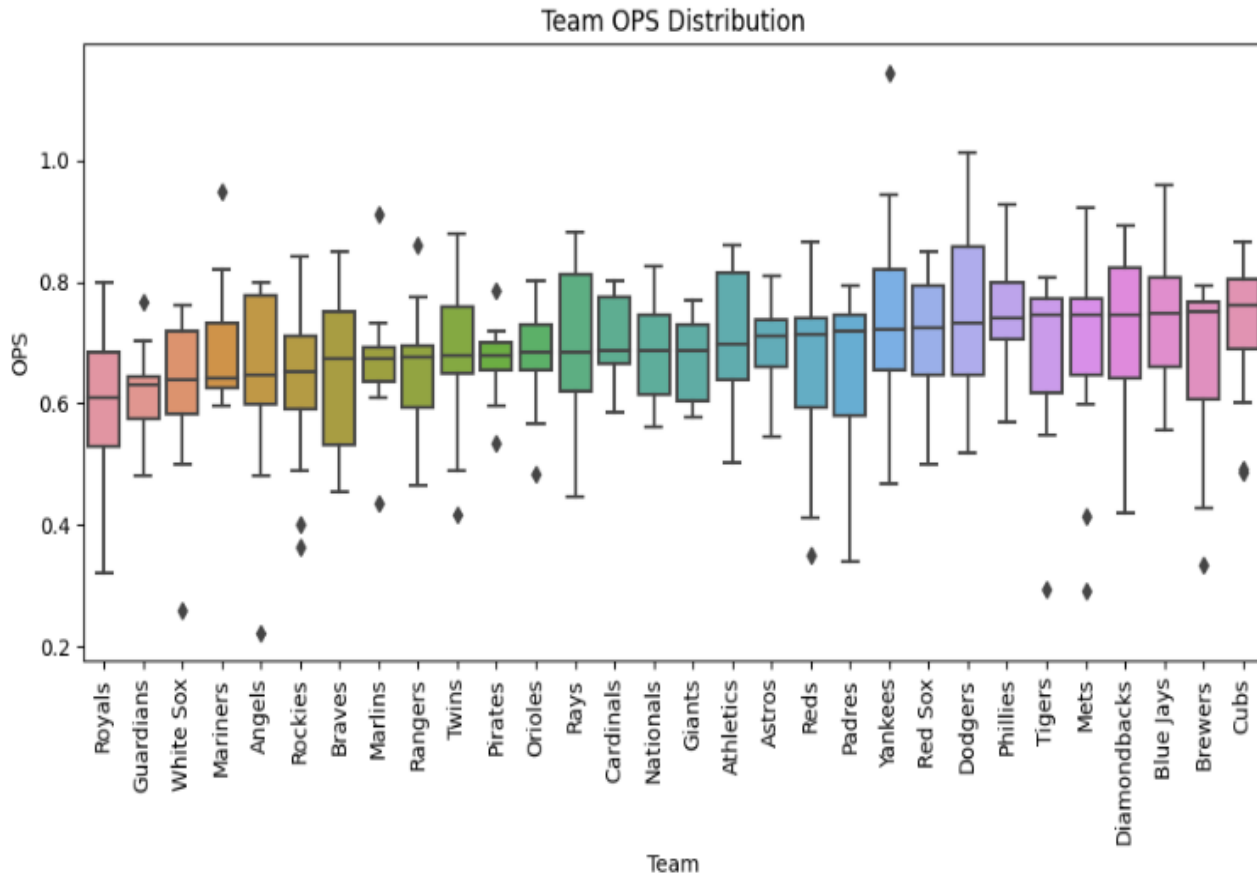


- Shows how each variable correlates to each other
- Shows how each variable correlates to Salary
- Good to know for deciding what to include in regressions

```
corr = mlb_data2[["OPS", "WAR", "HR", "RBI", "BattingAvg", "OFF", "Salary"]].corr()
sns.heatmap(corr, cmap="coolwarm", annot=True, fmt='.2f') #plot heatmap with num
plt.title("Correlation Heatmap of MLB Performance and Salary")
plt.tight_layout()
plt.savefig('heatmap.png', dpi=300)
plt.show()
```

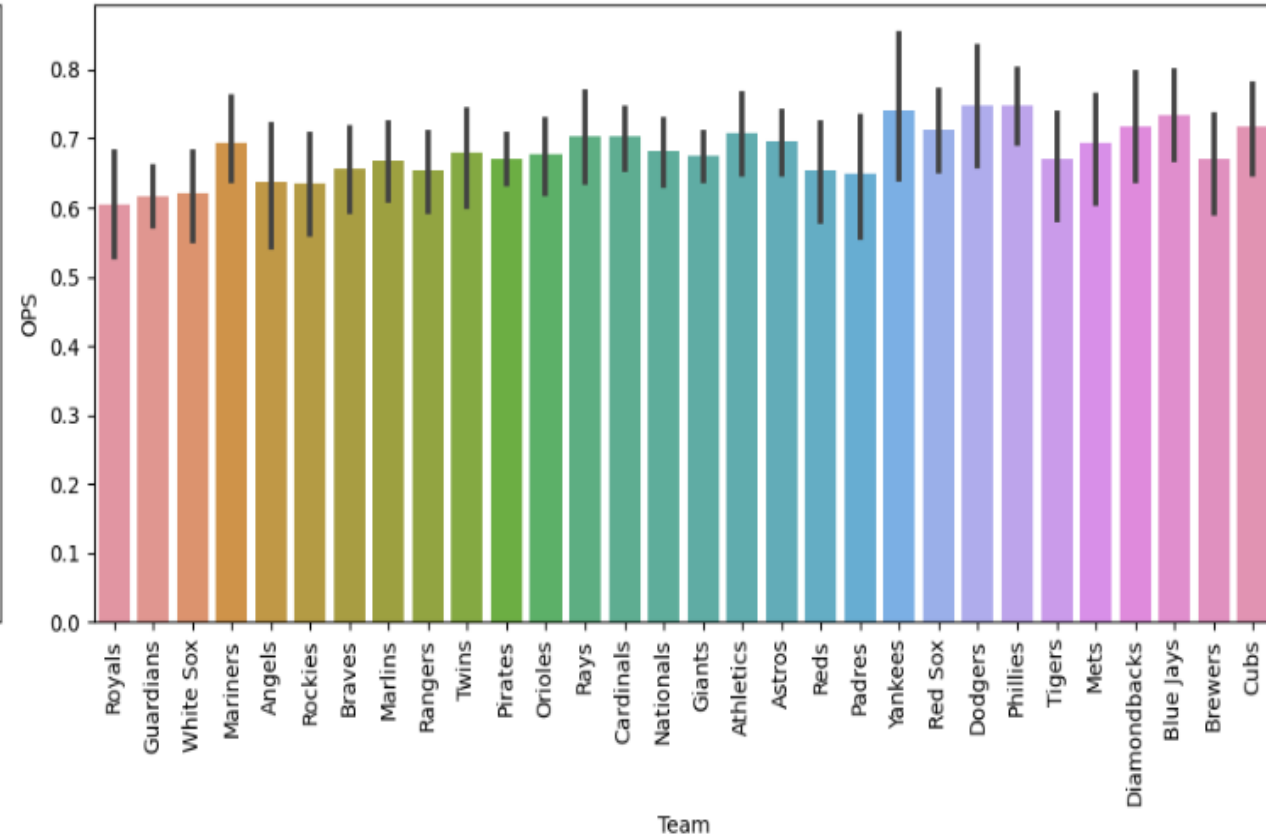
HOW OPS DIFFERS BY TEAM

OPS by Teams



Shows distribution of OPS data in each team

Average OPS by Team



Shows average OPS data in each team

```
## Residuals:
##      Min       1Q   Median       3Q      Max
## -15.737  -4.521   0.000   4.040  23.479
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    17.0822     5.9722   2.860 0.005480 **
## WAR             3.2522     0.7597   4.281 5.44e-05 ***
## TeamAngels      2.4497     6.9242   0.354 0.724487
## TeamAstros     -6.8815     6.4099  -1.074 0.286452
## TeamAthletics  -3.8532     6.1668  -0.625 0.533978
## TeamBlue Jays  -9.6060     5.8669  -1.637 0.105751
## TeamBraves     -9.1925     7.1221  -1.291 0.200774
## TeamBrewers    -7.1492     6.0252  -1.187 0.239156
## TeamCardinals  -6.1805     7.2332  -0.854 0.395574
## TeamCubs       -6.5583     5.3651  -1.222 0.225382
## TeamDiamondbacks -5.8694     6.0244  -0.974 0.333050
## TeamDodgers    -6.9834     6.0751  -1.150 0.253992
## TeamGiants     -1.9303     6.4852  -0.298 0.766799
## TeamGuardians -18.1153     9.6599  -1.875 0.064641 .
## TeamMariners    4.4636     6.3391   0.704 0.483523
## TeamMarlins    -8.2799     6.9130  -1.198 0.234798
## TeamMets        1.5976     5.9783   0.267 0.790018
## TeamNationals   0.4689     7.7120   0.061 0.951678
## TeamOrioles    -12.0673     7.1247  -1.694 0.094467 .
## TeamPadres     -4.6625     6.9463  -0.671 0.504146
## TeamPhillies   -0.6754     6.1081  -0.111 0.912251
## TeamPirates    -5.2034    11.1619  -0.466 0.642443
## TeamRangers    -7.4825     6.3996  -1.169 0.246018
## TeamRays       -5.7288     6.4703  -0.885 0.378771
## TeamRed Sox    -6.7367     5.4692  -1.232 0.221896
## TeamReds       -4.7424     7.8070  -0.607 0.545387
## TeamRockies     1.0576     7.7917   0.136 0.892393
## TeamRoyals     -6.3519     7.8699  -0.807 0.422148
## TeamTigers     -2.0615     6.2995  -0.327 0.744387
## TeamTwins      -5.7827     6.7583  -0.856 0.394923
## Position1B      4.3391     3.5003   1.240 0.218981
## Position2B     -9.0370     3.2500  -2.781 0.006856 **
## Position3B     -4.4676     3.1145  -1.434 0.155604
## PositionC      -1.1921     3.2415  -0.368 0.714085
## PositionDH      7.3264     5.0203   1.459 0.148642
## PositionDH/RHP 19.8593     9.8670   2.013 0.047737 *
## PositionINF    -12.6841     6.0525  -2.096 0.039486 *
## PositionLHP    -20.7703     5.7649  -3.603 0.000563 ***
## PositionRHP    -21.9006     4.2250  -5.184 1.78e-06 ***
## PositionSS     -7.8676     2.7605  -2.850 0.005641 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 8.501 on 75 degrees of freedom
## Multiple R-squared:  0.6788, Adjusted R-squared:  0.5117
## F-statistic: 4.063 on 39 and 75 DF,  p-value: 9.645e-08
```

DOES TEAM YOU PLAY ON AFFECT HOME RUNS

- Omitted team is Yankees
- Omitted position in Outfielder
- Meaningful coefficients are WAR and positions: 2nd, DH/RHP, INF, Pitchers, and Shortstop
- Shows that teams are mostly not statistically significant suggesting team affiliation does not strongly impact home runs
- Position relative to outfield and WAR hold significance
- Model explains 51% of home run variation
- Important to note that this is a sample with a little over 100 players in it so conclusions limit generalization

```
model15 <-lm(HR ~ WAR + Team + Position, data = data2)
summary(model15)
```

WHICH STATS MATTER MORE IN DETERMINING SALARY: TRADITIONAL OR ADVANCED

- Only significant variables are RBI in first model and WAR in second
- The traditional statistic model explains about 16% of variation in log salary
- Advanced statistics model performs bad (negative adjusted R squared)
- OPS does not have a negative correlation with Salary; it does not provide any additional information when WAR is controlled for
- In the combined model, explanatory power declines as multicollinearity increases
- Results show salary is not strongly determined by either traditional or advanced statistics
- A lot of variation in salary is determined by factors not in my model

```
model6 <- lm(log(Salary) ~ HR + BattingAvg + RBI + Position, data = data1) # traditional stats only
model7 <- lm(log(Salary) ~ WAR + OPS + Position, data = data2) #advanced stats only
model8 <- lm(log(Salary) ~ HR + BattingAvg + RBI + WAR + Position, data = data2) # advanced plus traditional stats

stargazer(model6, model7, model8,
  type = "text",
  title = "Regression Results",
  column.labels = c("Traditional Stats Model", "Advanced Stats Model", "Advanced + Traditional Stats"), #name each r
  regression column
  dep.var.labels = c("log(Salary)", "Log(Salary)", "Log(Salary)"),
  digits = 3) #how many decimals are shown
```

Dependent variable: log(Salary)

	Traditional Stats Model (1)	Advanced Stats Model (2)	Advanced + Traditional Stats (3)
HR	0.009 (0.016)		0.014 (0.027)
BattingAvg	0.377 (1.831)		-2.075 (1.526)
RBI	0.012** (0.006)		0.009 (0.012)
WAR		0.190* (0.112)	0.093 (0.121)
OPS		-0.430 (0.764)	
Position1B	0.009 (0.227)	0.254 (0.518)	0.059 (0.525)
Position2B	-0.033 (0.221)	0.128 (0.500)	0.382 (0.512)
Position3B	0.228 (0.231)	-0.059 (0.482)	0.111 (0.483)
PositionC	-0.067 (0.193)	-0.148 (0.483)	-0.061 (0.477)
PositionDH	0.430 (0.307)	0.054 (0.763)	-0.235 (0.762)
PositionDH/RHP	1.356 (1.243)	1.234 (1.517)	0.909 (1.553)
PositionINF	-0.562** (0.257)	-1.047 (0.774)	-0.518 (0.807)
PositionLHP		0.299 (1.061)	1.162 (1.110)
PositionRHP	-0.578 (1.185)	-0.101 (0.726)	0.865 (0.850)
PositionSS	0.151 (0.218)	0.014 (0.430)	0.219 (0.442)
Constant	13.953*** (0.384)	14.890*** (0.698)	14.393*** (0.742)
Observations	371	115	115
R2	0.187	0.080	0.129
Adjusted R2	0.160	-0.029	0.007
Residual Std. Error	1.178 (df = 358)	1.439 (df = 102)	1.414 (df = 100)
F Statistic	6.858*** (df = 12; 358)	0.735 (df = 12; 102)	1.053 (df = 14; 100)



THANK YOU

QUESTIONS